

TP03 – Lista Invertida

Alunos participantes: Isabel Cristina, Laura Bargas, Pedro Mattar e Yuri Penido.

O sistema de busca por palavras-chave (TP03) é uma evolução do sistema acadêmico para gestão de cursos livres da PUC Minas. Esta versão implementa um Índice Invertido associado ao modelo estatístico TFxIDF (Term Frequency — Inverse Document Frequency), permitindo que os usuários realizem buscas textuais inteligentes pelos nomes dos cursos livres disponíveis no ecossistema, retornando os resultados ordenados por relevância.

O sistema utiliza persistência em arquivos binários de acesso aleatório (RandomAccessFile) , estendendo as estruturas de dados avançadas já existentes (Tabelas Hash Extensíveis e Árvores B+) com a inclusão de arquivos de Listas Invertidas para indexação e busca de termos.

Link do vídeo: <https://youtu.be/TT9y1vTKzuc>

Estrutura de Classes Criadas

1. Novas Classes Utilizadas (TP3)

- **ElementoLista:** Entidade que encapsula os dados fundamentais de indexação de cada termo, armazenando o atributo id (referente ao ID do curso) e o atributo frequencia (referente à taxa de ocorrência do termo naquele curso específico — TF).

```

public class ElementoLista implements Comparable<ElementoLista>, Cloneable {

    private int id;
    private float frequencia;

    public ElementoLista(int i, float f) {
        this.id = i;
        this.frequencia = f;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public float getFrequencia() {
        return frequencia;
    }

    public void setFrequencia(float frequencia) {
        this.frequencia = frequencia;
    }

    @Override
    public String toString() {
        return "("+this.id+";"+this.frequencia+");";
    }
}

```

- **ListasInvertidas:** Estrutura em disco responsável pelo CRUD de termos, que gerencia de forma indexada o arquivo de dicionário (arqDicionario) e o arquivo de blocos encadeados (arqBlocos), manipulando diretamente arrays e instâncias de objetos do tipo ElementoLista.

```

public class ListaInvertida {
    class Bloco {
        public boolean create(ElementoLista e) {
            int i = quantidade - 1;
            while (i >= 0 && e.getId() < elementos[i].getId()) {
                elementos[i + 1] = elementos[i];
                i--;
            }
            i++;
            elementos[i] = e.clone();
            quantidade++;
            return true;
        }

        // Testa se um valor existe no bloco
        public boolean test(int id) {
            if (empty())
                return false;
            int i = 0;
            while (i < quantidade && id > elementos[i].getId())
                i++;
            if (i < quantidade && id == elementos[i].getId())
                return true;
            else
                return false;
        }

        // Lê um valor existente no bloco
        public ElementoLista read(int id) {
            if (empty())
                return null;
            int i = 0;

```

2. Modificações em Classes Existentes (Integração)

- ArquivoCurso:** Modificado para sincronizar o índice textual de forma imediata e transparente. Sempre que um curso é incluído, o método incrementaEntidades() da lista invertida é acionado e as palavras do nome do curso são processadas e persistidas via ListaInvertida.create(). Em operações de exclusão, executa o método decrementaEntidades() e limpa as referências via ListaInvertida.delete(). Em atualizações, sincroniza os termos alterados através do método ListaInvertida.update(). Além disso, a criação STOPWORDS também foi implementada.

```

public class ArquivoCurso extends Arquivo<Curso> {

    HashExtensivel<ParCodigoId> indiceCodigo;
    ArvoreBMais<ParNomeId> indiceNome;
    ArvoreBMais<ParIdId> indiceUsuarioCurso;
    ListaInvertida indiceInvertido;

    private static final Set<String> STOPWORDS = new HashSet<String>() {{
        String[] words = new String[] {
            "a", "o", "as", "os", "um", "uma", "uns", "umas",
            "de", "do", "da", "dos", "das", "em", "no", "na", "nos", "nas",
            "para", "por", "e", "com", "sem", "ao", "aos", "como", "que",
            "quem", "onde", "quando", "se", "sao", "ser", "ter", "sob", "sobre",
            "entre", "ate", "ou", "mas", "so", "sua", "seu", "seus", "suas",
            "num", "numa", "nao", "nao", "nao", "pelo", "pela", "pelos", "pelas",
            "este", "esta", "estes", "estas", "aquele", "aquela", "aqueles", "aquelas",
            "isto", "isso", "aquilo", "me", "mim", "comigo", "te", "ti", "contigo",
            "lhe", "lhes", "nos", "vos", "eles", "elas", "ele", "ela", "eu", "tu",
            "por", "sobre", "entre", "sob", "através", "apos", "antes", "durante",
            "quando", "onde", "porque", "porquê", "porq", "peloque", "como", "qual", "quais",
            "seu", "sua", "meu", "minha", "nosso", "nossa", "vosso", "vossa",
            "já", "ja", "ainda", "tambem", "também", "mais", "menos", "muito", "muitos", "muita", "muitas",
            "cada", "todo", "todos", "toda", "todas", "algum", "alguma", "alguns", "algumas", "nenhum", "nenhuma",
            "haja", "ha", "houve", "foi", "era", "são", "sera", "será", "está", "esta", "estao", "estavam",
            "comigo", "consigo", "sem", "sob", "per", "pela", "pelas", "pelos", "ao", "aos",
            "dos", "das", "num", "numa", "nos", "na", "no", "nao", "sim", "nao", "nao",
            "-", ""};
        for(String w : words) add(w);
    }};
}

```

```

@Override
public int create(Curso c) throws Exception {
    // garante código único
    if(c.getCodigoCompartilhavel() == null || c.getCodigoCompartilhavel().length()==0) {
        String codigo;
        do {
            codigo = Curso.gerarCodigoCompartilhavel();
        } while(indiceCodigo.read(Math.abs(codigo.hashCode()))!=null);
        c.setCodigoCompartilhavel(codigo);
    } else {
        while(indiceCodigo.read(Math.abs(c.getCodigoCompartilhavel().hashCode()))!=null) {
            c.setCodigoCompartilhavel(Curso.gerarCodigoCompartilhavel());
        }
    }

    int id = super.create(c);
    indiceCodigo.create(new ParCodigoId(c.getCodigoCompartilhavel(), id));
    indiceNome.create(new ParNomeId(c.getNome(), id));
    indiceUsuarioCurso.create(new ParIdId(c.getIdUsuario(), id));
    // atualiza indice invertido: palavras do nome
    Map<String, Float> tf = computeTF(c.getNome());
    for(Map.Entry<String, Float> en : tf.entrySet()) {
        try { indiceInvertido.create(en.getKey(), new ElementoLista(id, en.getValue())); } catch(Exception ex) {}
    }
    return id;
}

```

- **ControleInscricao:** Gerencia a lógica de negócio na opção de menu de busca por palavras-chave. Ele processa a string digitada pelo usuário, recupera os arrays de ElementoLista gerados pela busca em disco através do método

ListalInvertida.read(), calcula o valor ponderado de IDF em tempo de execução com base no número total de entidades do dicionário, consolida as notas através da soma acumulada de elementos que possuem o mesmo ID e gera o ranking ordenado de forma decrescente.

```
else if(ch=='B' || ch=='b') {
    System.out.print(s: "Digite termos de busca: ");
    String consulta = console.nextLine();
    if(consulta.length()==0) continue;
    Curso[] encontrados = arqCursos.searchByKeywords(consulta);
    if(encontrados.length==0) {
        System.out.println(x: "Nenhum curso encontrado para os termos informados.");
        System.out.println();
        System.out.println(x: "Pressione ENTER para continuar...");
        console.nextLine();
        continue;
    }
}
```

- **VisaolInscricao:** Interface de terminal atualizada para capturar a string de consulta por termos na opção correspondente e exibir a listagem final de cursos gerada pelo controle em sua ordem precisa de pontuação e relevância.

```
// tentará inscrever
if(c.getEstado()!=0) {
    System.out.println(x: "ALERTA: Este curso nao aceita inscricoes.");
    System.out.println();
    System.out.println(x: "Pressione ENTER para continuar...");
    console.nextLine();
    continue;
}
```

```
else if(ch=='B' || ch=='b') {
    System.out.print(s: "Digite termos de busca: ");
    String consulta = console.nextLine();
    if(consulta.length()==0) continue;
    Curso[] encontrados = arqCursos.searchByKeywords(consulta);
    if(encontrados.length==0) {
        System.out.println(x: "Nenhum curso encontrado para os termos informados.");
        System.out.println();
        System.out.println(x: "Pressione ENTER para continuar...");
        console.nextLine();
        continue;
    }
}
```

Operações Especiais Implementadas

- **Processamento e Normalização de Termos:** Utilização do fatiamento de strings (split) associado ao método estático ParNomeId.transforma(String str)

para quebrar o nome de cada curso em um vetor de tokens em letras minúsculas e sem acentuação, aplicando em seguida um filtro para descartar conectivos irrelevantes (*stop words*).

```
@Override
public boolean update(Curso novoCurso) throws Exception {
    Curso c = read(novoCurso.getID());
    if(c==null)
        return false;
    if(super.update(novoCurso)) {
        if(c.getCodigoCompartilhavel().compareTo(novoCurso.getCodigoCompartilhavel())!=0) {
            indiceCodigo.delete(Math.abs(c.getCodigoCompartilhavel().hashCode()));
            indiceCodigo.create(new ParCodigoId(novoCurso.getCodigoCompartilhavel(), novoCurso.getID()));
        }
        if(c.getNome().compareTo(novoCurso.getNome())!=0) {
            indiceNome.delete(new ParNomeId(c.getNome(), c.getID()));
            indiceNome.create(new ParNomeId(novoCurso.getNome(), novoCurso.getID()));
            // atualizar indice invertido: comparar termos antigos e novos
            Map<String, Float> antigos = computeTF(c.getNome());
            Map<String, Float> novos = computeTF(novoCurso.getNome());
            // remover termos que ficaram
            for(String t : antigos.keySet()) {
                if(!novos.containsKey(t)) {
                    try { indiceInvertido.delete(t, novoCurso.getID()); } catch(Exception ex) {}
                }
            }
            // adicionar / atualizar termos novos
            for(Map.Entry<String, Float> en : novos.entrySet()) {
                String t = en.getKey();
                float tfv = en.getValue();
            }
        }
    }
}
```

- **Cálculo Dinâmico de IDF:** O cálculo do peso estatístico de cada termo — $\text{IDF} = \log_{10}(\text{Total de Cursos} / \text{Cursos com o Termo}) + 1$ — utiliza o método `numeroEntidades()` da classe `ListaInvertida` em tempo de execução, garantindo a calibração precisa dos pesos dos termos no sistema após modificações no arquivo.

```
private Map<String, Float> computeTF(String texto) {
    Map<String, Integer> counts = new HashMap<>();
    if(texto==null) return new HashMap<String, Float>();
    // normalizar: remover acentos, toLower, manter letras e numeros
    String s = Normalizer.normalize(texto, Normalizer.Form.NFD).replaceAll(regex: "\\p{M}", replacement: "");
    s = s.toLowerCase().replaceAll(regex: "[^a-z0-9\\s]", replacement: " ");
    String[] parts = s.split(regex: "\\s+");
    int total = 0;
}
```

- **Ranking Acumulativo de Múltiplos Termos:** Ao realizar buscas com múltiplas palavras-chave, o sistema varre as listas de cada termo e efetua a soma dos scores parciais de $\text{TF} \times \text{IDF}$ para registros com IDs coincidentes, fazendo com que cursos com maior nível de correspondência textual fiquem posicionados organicamente no topo da listagem.

```
String[] parts = s.split(regex: "\\s+");
int total = 0;
for(String p : parts) {
    if(p==null) continue;
    p = p.trim();
    if(p.length()==0) continue;
    if(STOPWORDS.contains(p)) continue;
    String stem = stemWord(p);
    if(stem.length()==0) continue;
    if(STOPWORDS.contains(stem)) continue;
    counts.put(stem, counts.getDefault(stem, defaultValue: 0) + 1);
    total++;
}
```

Checklist:

1. O índice invertido com os termos dos nomes dos cursos foi criado usando a classe ListaInvertida?
Sim.
2. É possível buscar cursos por palavras no menu de inscrição?
Sim.
3. O trabalho compila corretamente?
Sim.
4. O trabalho está completo e funcionando sem erros de execução?
Sim.
5. O trabalho é original e não a cópia de um trabalho de outro grupo?
Sim.